User Registration (Backend + Client)

This project uses a **lightweight registration flow** backed by the **UserProfile** model stored in **MongoDB** (via **django-mongodb-backend**).

Where it lives

- **Backend URLs**: backend/config/urls.py mounts users at api/v1/users
- **Users app**: backend/apps/users/
  - models.py: UserProfile
  - views.py: CreateUserView, LoginView, ModifyUserView
  - urls.py: routing for create/login/modify
- **Client auth API wrapper**: client/src/features/auth/api.ts

Data model: UserProfile

The registration flow persists a **UserProfile** document with these fields:

- **Identity**: first_name, last_name, email
- **Optional profile**: phone, dob, location (JSON)
- **Personalization attributes**: user_attributes (JSON)
- **Onboarding marker**: onboarded (boolean)
- **Timestamps**: created_at

Source: **backend/apps/users/models.py**.

Base URL

All user endpoints are served by Django at:

- **Local dev**: http://localhost:8000/api/v1/users

In the client, **API_BASE_URL** is typically **/api/v1** and requests go to:

- /api/v1/users/... (often through a Vite proxy in development)

Source: **client/src/lib/api.ts**, **client/src/features/auth/api.ts**.

Endpoints

1) Create user (registration)

- **Method**: POST
- **Path**: /api/v1/users/
- **Purpose**: Create a new UserProfile with minimal identity fields.

Request

```
{
  "first_name": "John",
  "last_name": "Smith",
  "email": "johnsmith@gmail.com"
}
```

Success (201)

```
{
  "status": "success",
  "data": {
    "id": "699677776d39e28e8af72ad5",
    "onboarded": false,
    "createdAt": "2026-02-19T02:37:43Z"
  }
}
```

Notes

- email is **unique** at the database/model level. Attempting to create a duplicate email will error.
- This endpoint does **not** set a password (see “Auth note” below).

Backend source: **backend/apps/users/views.py** (**CreateUserView**).

2) Login (lookup by email)

- **Method**: POST
- **Path**: /api/v1/users/login/
- **Purpose**: Find a user by email and return the full profile record.

Request

```
{
  "email": "johnsmith@gmail.com"
}
```

Success (200)

```
{
  "status": "success",
  "data": {
    "id": "699677776d39e28e8af72ad5",
    "first_name": "John",
    "last_name": "Smith",
    "email": "johnsmith@gmail.com",
    "phone": "",
    "dob": null,
    "location": {},
    "user_attributes": {},
    "onboarded": false,
    "created_at": "2026-02-19T02:37:43.000000+00:00"
  }
```

}

Error cases

- **400**: email missing
- **404**: no user exists with that email

Backend source: **backend/apps/users/views.py** (**LoginView**).

3) Modify user (finish onboarding / update profile)

- **Method**: PUT
- **Path**: /api/v1/users/<user_id>/
- **Purpose**: Update profile fields and set onboarded=true.

Request (example)


```
{
  "first_name": "John",
  "last_name": "Smith",
  "phone": "1234567890",
  "dob": "1990-05-20",
  "location": {
    "country": "United States",
    "state": "Arizona",
    "county": "Maricopa County"
  },
  "user_attributes": {
    "profession": "Education Professional",
    "employment_type": "Full-Time",
    "income_bracket": "$35,000 - $49,999"
  }
}
```


Success (200)


```
{
  "status": "success",
  "data": { "id": "699677776d39e28e8af72ad5" }
}
```


Validation

user_attributes is validated for a fixed set of enumerations:

- profession
- employment_type
- veteran_status

- disability_status
- marital_status
- income_bracket

If an invalid value is provided, the API returns **400** with **errors** keyed by field.
Backend source: **backend/apps/users/views.py** (**ModifyUserView** + **validate_enum_attributes**).

Client usage

The client calls:

- createUser({ firstName, lastName, email }) → POST /api/v1/users/
- loginUser(email) → POST /api/v1/users/login/

Source: **client/src/features/auth/api.ts**.

Auth note (important)

This codebase currently treats “login” as a **profile lookup by email** (no password, no JWT/session). If you later add real authentication (passwords, OTP, JWT, OAuth), this document should be updated along with:

- endpoint contracts
- client auth flow
- any middleware/permissioning around chat/document APIs